Data Mining & Machine Learning Project Report: Linear Regression, Decision Trees and KNN

MSc in Software Design with AI | Name: Sampath Reddy Kamsani | Student ID: A00348851

Dataset documentation: (1) Auto MPG: https://archive.ics.uci.edu/dataset/9/auto+mpg

(2) Heart Disease (Cleveland): https://archive.ics.uci.edu/dataset/45/heart+disease. CSV copies are in the data folder.

1. Linear Regression: Car Fuel Efficiency (Auto MPG)

1.1 Objectives

A fleet buyer or importer wants an early estimate of a car's fuel economy from its specification, before costly road tests, because fuel use drives running cost and emissions compliance. I also test whether a polynomial term improves the model.

Target variable: mpg (miles per gallon, continuous). Predictors: cylinders, displacement, horsepower, weight, acceleration, model year, origin. Success: test RMSE of about 3 mpg or less and R^2 above 0.8.

1.2 Data Exploration

398 cars, 9 columns. Only horsepower has missing values (6 rows), which I filled with the median. The identifier column name was dropped, and origin was one-hot encoded.

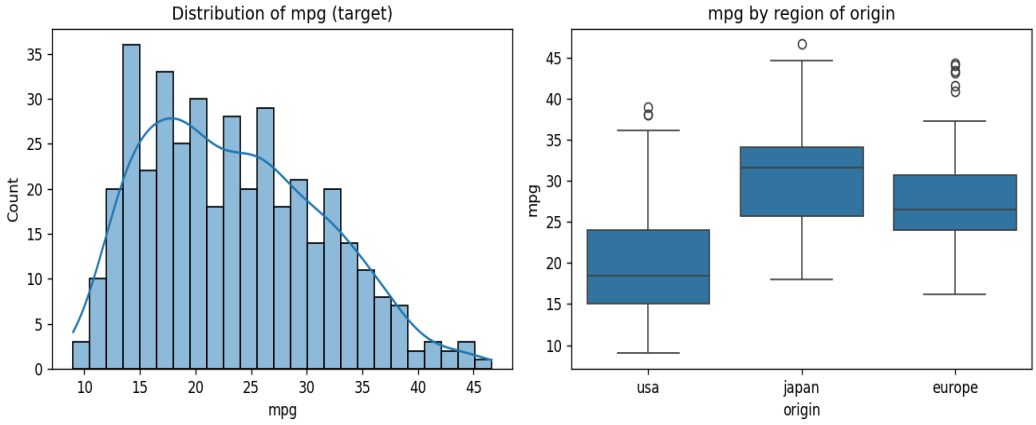


Figure 1: distribution of mpg and mpg by origin

Comment: *mpg is mildly right-skewed (skew 0.46), mostly between 15 and 30, so no transformation is needed. US cars average about 20 mpg against 28 for Europe and 30 for Japan, so origin is a useful predictor.*

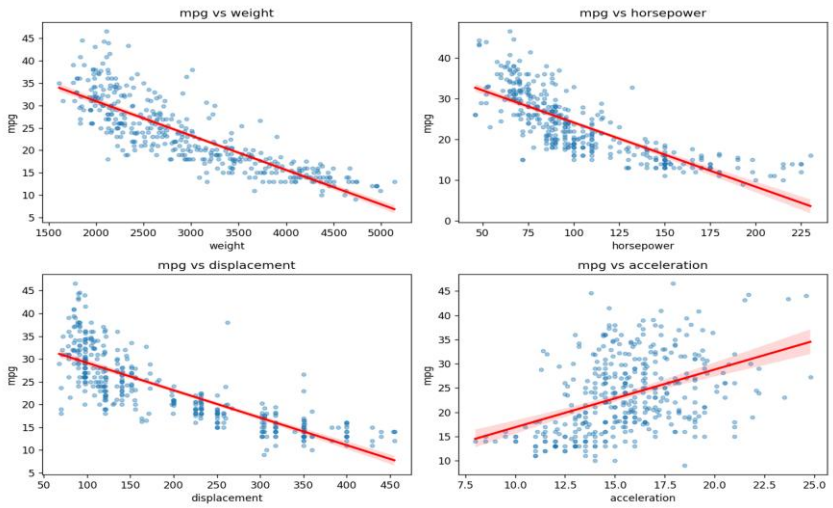


Figure 2: mpg vs key features

Comment: *Figure 2: weight and horsepower have strong negative relationships with mpg, but the weight relationship is visibly curved (points sit above the red line at both ends), which motivates a squared term. Figure 3: weight, displacement, cylinders and horsepower correlate at -0.77 to -0.83 with mpg and at 0.84 to 0.95 with each other, so multicollinearity is expected. Individual coefficients need care, but prediction is not much affected. model_year (+0.58) shows efficiency improving over time.*

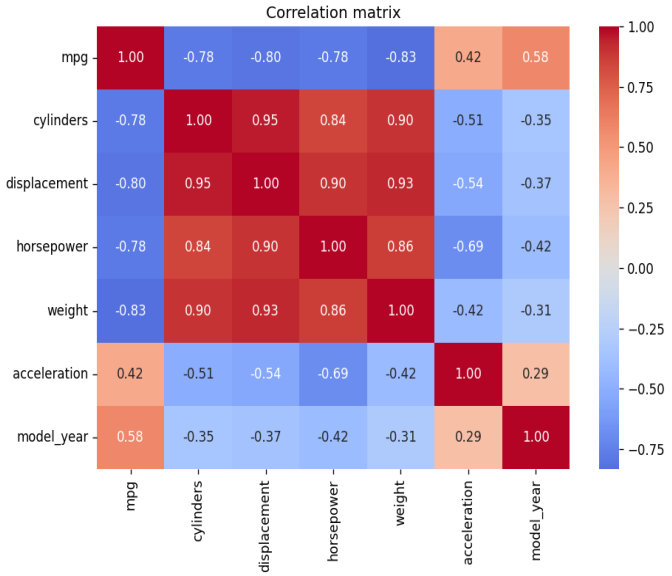


Figure 3: correlation matrix

1.3 Modeling

```
X = df.drop(columns='mpg'); y = df['mpg']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
m2 = LinearRegression().fit(X_train, y_train)                # Model 2: all features
cross_val_score(LinearRegression(), X_train, y_train, cv=5, scoring='r2')
X_train['weight_sq'] = (X_train['weight']/1000)**2           # polynomial term (Models 3 and 4)
```

Model 2 (all features) has intercept -18.91 and coefficients: model_year +0.826, origin_usa -2.95, origin_japan -0.28 (reference: Europe), weight -0.0070, horsepower -0.0146, acceleration +0.068, cylinders -0.17, displacement +0.020. Each later model year adds about 0.8 mpg, every extra 100 lb costs about 0.7 mpg and, all else equal, US cars are about 2.9 mpg lower. The tiny or wrong-signed cylinders and displacement values reflect collinearity.

Polynomial regression: a quadratic is appropriate here. Model 3 (weight + weight², weight in 1000 lb) gives mpg = 62.39 - 18.41·w + 1.67·w², and Model 4 adds weight² (coefficient +2.28) to all features. Figure 4 shows weight-only CV R² peaking at degree 2 (0.691, versus 0.674 for a straight line) and not improving at higher degrees, so I stop at degree 2.

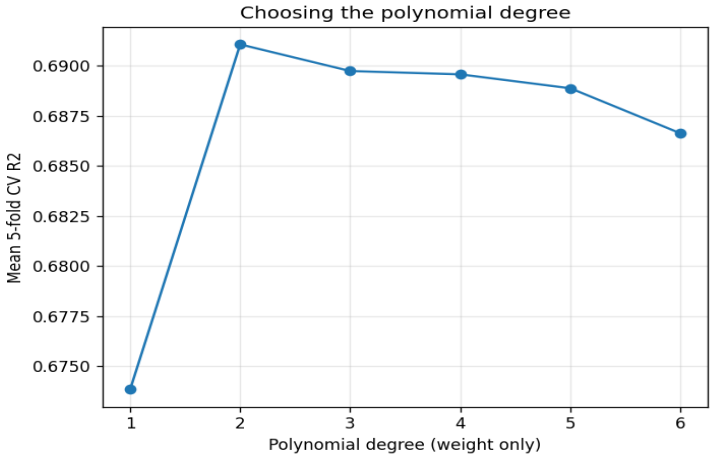


Figure 4: CV R² against polynomial degree

1.4 Evaluation & Conclusion

Model	CV R^2 (5-fold)	CV RMSE	Test R^2	Test RMSE
1: weight only	0.674	4.48	0.723	3.86
2: all features	0.805	3.46	0.845	2.89
3: weight + weight²	0.691	4.34	0.755	3.63
4: all features + weight²	**0.840**	**3.12**	**0.882**	**2.52**

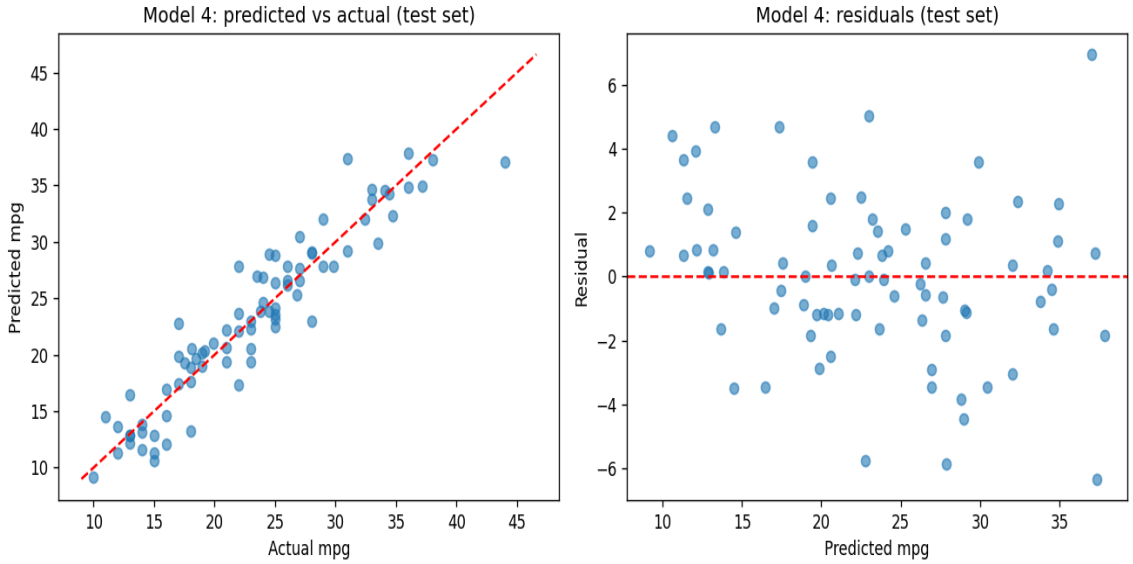


Figure 5: Model 4 predicted vs actual and residuals (test set)

Comment: *predictions follow the diagonal closely and the residuals show no strong pattern, with slightly wider spread for low-mpg (heavy) cars.*

The squared term improves both the simple model (test R^2 0.72 to 0.76) and the full model (CV R^2 0.805 to 0.840, test R^2 0.845 to 0.882). Model 4 meets the objective: it predicts mpg to within about 2.5 mpg on unseen cars, and CV and test results agree, so overfitting is unlikely. Limitations: the data covers 1970 to 1982 only and the test set has 80 cars, so figures are somewhat noisy.

2. Decision Trees: Heart Disease Diagnosis

2.1 Objectives

A cardiology clinic wants to prioritise patients for expensive follow-up tests using routine clinical measurements. Missing a sick patient is costlier than an extra check, so recall on the disease class matters alongside accuracy, and the model should be readable by a clinician.

Target variable: disease (1 = heart disease present, 0 = absent), using 13 clinical predictors (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal). Success: test accuracy well above the majority baseline (54%) with reasonable recall.

2.2 Data Exploration

303 patients, no missing values, one duplicate row removed (302 left: 164 healthy, 138 diseased). Important finding: in this CSV, target = 1 corresponds to healthy patients (higher max heart rate 158 vs 139, lower ST depression 0.58 vs 1.59, fewer angina cases), so I created disease = 1 - target. Classes are fairly balanced, so accuracy is meaningful.

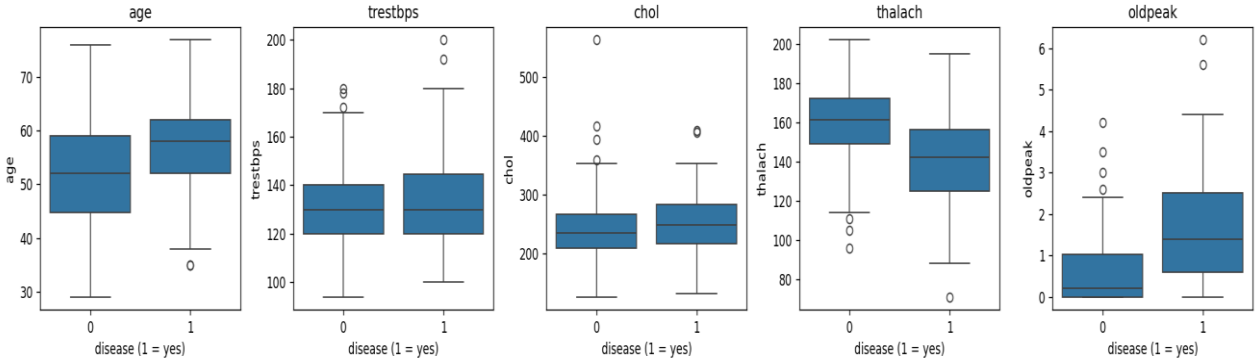


Figure 6: numeric predictors by disease status

Comment: *thalach is lower with disease (median 142 vs 161) and oldpeak is higher (1.4 vs 0.2); age is slightly higher (58 vs 52). chol and trestbps overlap heavily, so they are weak on their own. Outliers in chol are kept, since trees are not sensitive to them.*

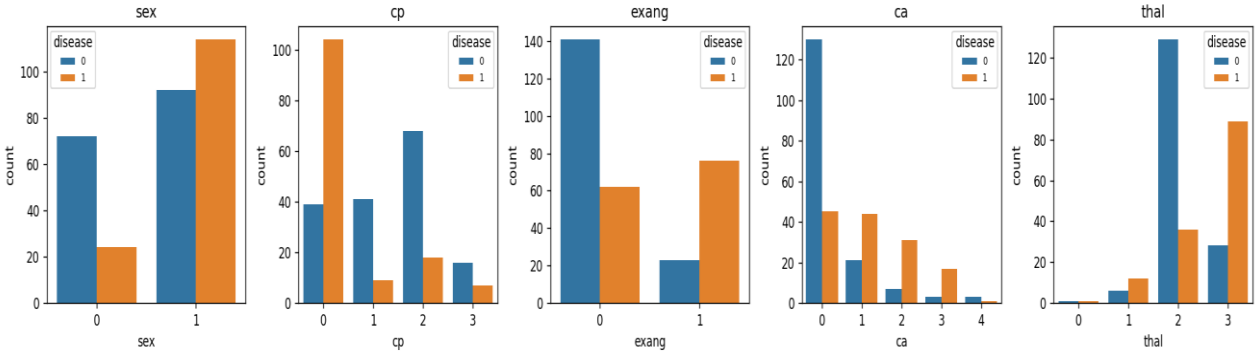


Figure 7: categorical predictors by disease status

Comment: *disease is more common in men (55% vs 25%), with exercise angina (77% vs 31%) and with more major vessels coloured (ca: 26% at 0, over 80% at 2 to 3). thal code 3 has 76% disease against 22% for code 2, and chest-pain code 0 has by far the highest rate (73%). So these categorical features should be strong splits.*

2.3 Modeling

```
X = df.drop(columns='disease'); y = df['disease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    {'max_depth': range(1, 11), 'min_samples_leaf': [1, 3, 5, 10]}, cv=5)
grid.fit(X_train, y_train);  tree = grid.best_estimator_
```

An unrestricted tree reaches depth 11 with 44 leaves, 100% training accuracy but only 75.1% cross-validated accuracy, which is overfitting. Grid search with 5-fold CV chose max_depth = 4 and min_samples_leaf = 5 (CV accuracy 0.813): depth 4, 13 leaves.

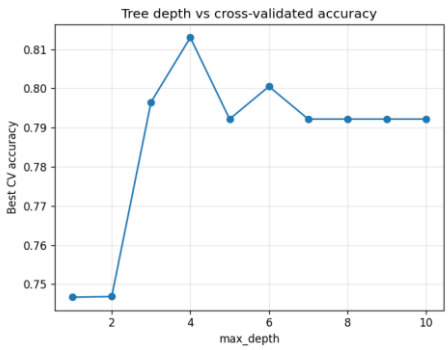


Figure 8: depth vs CV accuracy

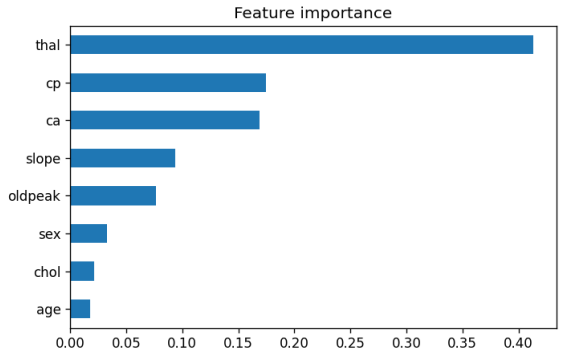


Figure 9: feature importance

Comment: *CV accuracy rises sharply up to depth 4 then drops and plateaus near 0.79, as deeper trees fit noise. Importance is led by thal (0.41), then cp (0.17) and ca (0.17), with slope and oldpeak smaller, matching the exploration.*

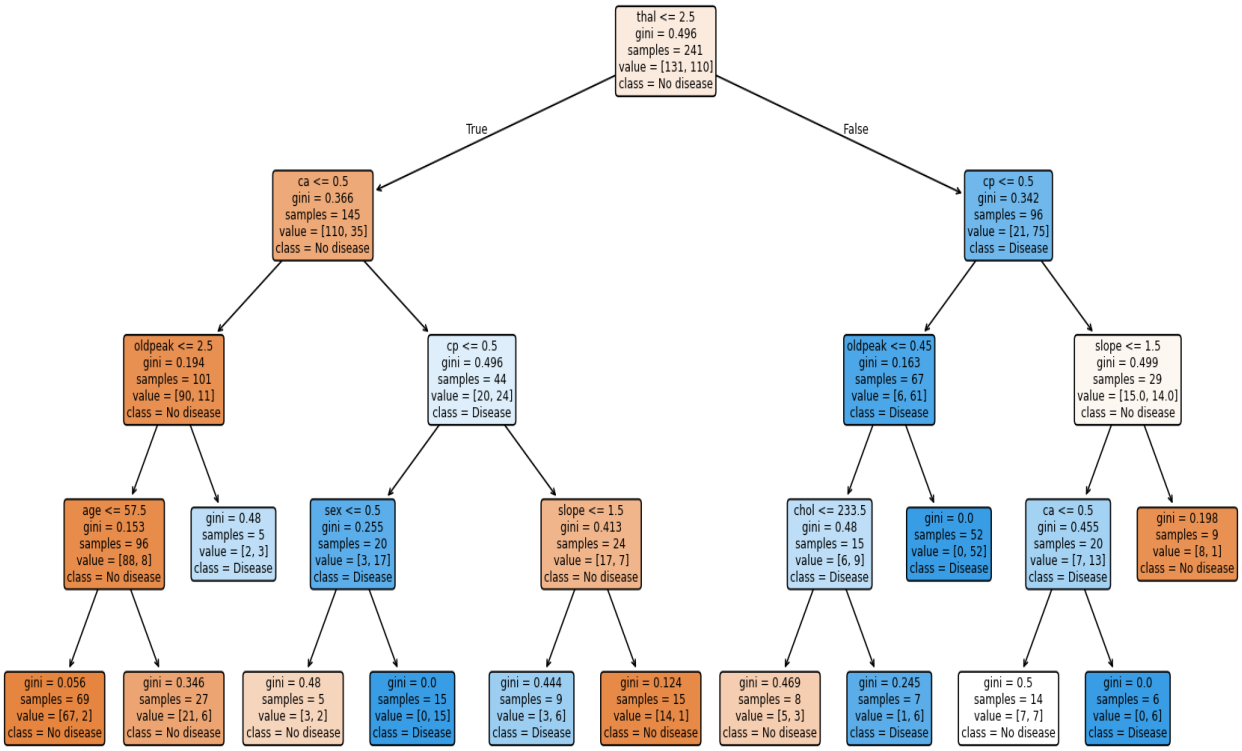


Figure 10: the final decision tree (depth 4)

2.4 Evaluation & Conclusion

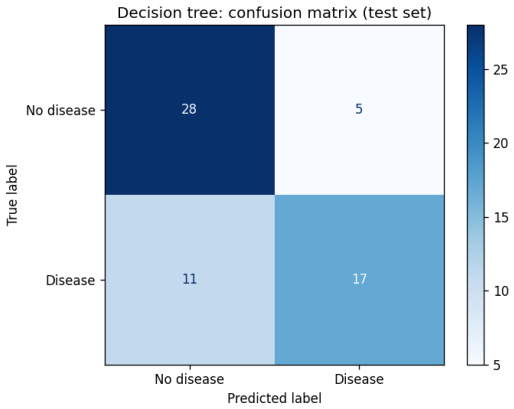


Figure 11: confusion matrix (test, n = 61)

Test accuracy is 0.738 (45 of 61) against the 0.541 baseline. For the disease class precision is 0.77 and recall 0.61; 11 of 28 sick patients were missed (false negatives) and 5 healthy patients were flagged. The tree beats the baseline by about 20 points with a few readable rules whose drivers (thal, cp, ca) make clinical sense. However, a recall of 61% is too low for stand-alone screening. Test accuracy is below the CV figure (0.81) partly because selecting parameters on CV scores is optimistic and partly because one patient equals 1.6 points in a 61-patient test set.

3. kNN: Heart Disease Diagnosis (same dataset)

3.3 Modeling

```
prep = ColumnTransformer([('num', StandardScaler(), ['age','trestbps','chol','thalach','oldpeak','ca']),
                          ('cat', OneHotEncoder(handle_unknown='ignore'), ['cp','restecg','slope','thal'])],
                         remainder='passthrough')
knn_pipe = lambda k: Pipeline([('prep', prep), ('knn', KNeighborsClassifier(n_neighbors=k))])
cv_acc = [cross_val_score(knn_pipe(k), X_train, y_train, cv=5).mean() for k in range(1, 31)]
```

The same cleaning and the same split (random_state = 42, stratified) are used, so both models face the identical 61 test patients. Scaling is essential: 5-fold CV accuracy at k = 5 is 0.639 on raw features but 0.842 once numeric features are standardised and nominal codes one-hot encoded, because otherwise chol and thalach dominate the distance.

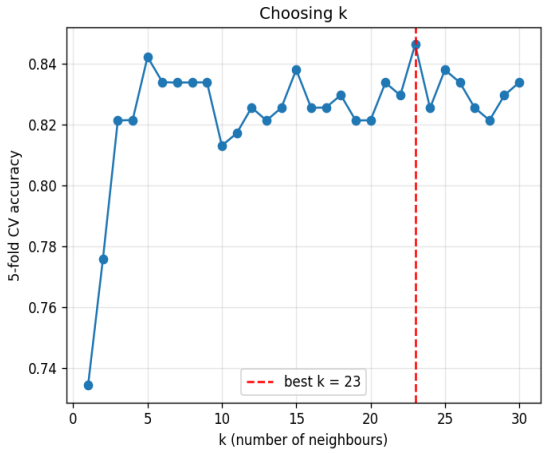


Figure 12: k vs CV accuracy

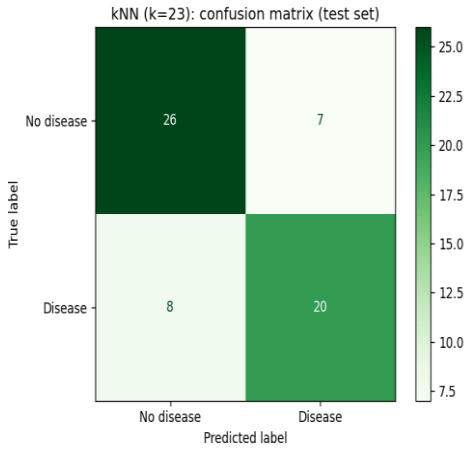


Figure 13: kNN confusion matrix (test)

Comment: *k = 1 overfits (about 0.73). Accuracy rises to k = 5 and then stays within about 0.81 to 0.85, peaking at k = 23 (0.846), only marginally above k = 5 (0.842). I keep k = 23 for a smoother boundary. The confusion matrix shows 20 of 28 sick patients found (8 missed) and 7 healthy patients flagged.*

3.4 Evaluation & Comparison with Decision Tree

Metric	Decision tree	kNN (k = 23)
10-fold CV accuracy (train)	0.776	0.817
Test accuracy	0.738	0.754
Test precision (disease)	0.773	0.741
Test recall (disease)	0.607	0.714
False negatives (missed cases)	11	8

kNN is better in cross-validation (+4 points) and slightly better on the test set (+1.6 points, a single patient, so not conclusive alone). The practical gain is recall: 8 missed patients instead of 11. The tree has marginally higher precision and is far more interpretable, whereas kNN offers no explanation and must store all training data. Neither reaches enough recall (best 71%) for unsupervised clinical use. Recommendation: use kNN if catching cases is the priority and the tree if explaining decisions matters more. Both could improve with more data, an ensemble such as a random forest, or a lower decision threshold to trade precision for recall.